Manual de integración

CAPTCHA Lógico Visual: Piedra, Papel o Tijeras

Cómo incorporar el CAPTCHA a cualquier programa en Python

Elaborado por Kevin Villalobos Obando

Versión 1.0

Octubre de 2026

Manual de integración del CAPTCHA Lógico Visual

Este manual explica cómo colocar el CAPTCHA lógico visual delante de cualquier programa escrito en Python, de modo que nadie llegue a la parte protegida sin superar antes el desafío. El CAPTCHA muestra una imagen de piedra, papel o tijeras y pide elegir, entre tres botones con las mismas imágenes, la opción que le gana. La serie se juega al mejor de tres, con quince segundos por ronda. Dos errores bloquean el acceso durante cincuenta segundos, y ese bloqueo se guarda en disco para que cerrar el programa no sirva para saltárselo.

Todo el mecanismo se reduce a una función, `verificar_acceso()`. Esa función abre la ventana del CAPTCHA, espera a que la serie termine y devuelve verdadero solo cuando la persona ganó. El programa que se quiere proteger no necesita conocer las reglas del juego: le basta con preguntar el resultado y decidir si continúa. En las siguientes secciones se describen dos maneras de hacerlo (importar el archivo `captcha.py` o pegar su código dentro del programa) y las reglas que hay que respetar para que la puerta de acceso funcione bien.

Figura 1

Ventana del CAPTCHA durante la primera ronda



Nota. Captura propia. Arriba se muestra el símbolo que hay que vencer; abajo, las tres opciones en un orden que cambia en cada ronda.

Requisitos

El CAPTCHA funciona con Python 3.8 o superior y usa Tkinter, que ya viene incluido en el instalador oficial de Python para Windows; en distribuciones de Linux se instala aparte con el paquete `python3-tk`. Pillow es opcional. Si está instalado, las imágenes se escalan con mejor calidad, y si no lo está, el CAPTCHA recurre al lector PNG propio de Tkinter sin que el programa falle. No se necesita ningún archivo de imagen aparte, porque las tres imágenes viajan dentro del código, codificadas en base64.

Cómo funciona la puerta de acceso

La integración siempre sigue el mismo orden: CAPTCHA, validación y acceso concedido. Cuando el programa llega a la línea que llama a `verificar_acceso()`, la ejecución se queda detenida ahí mientras la ventana del CAPTCHA está abierta. Al terminar la serie, la función devuelve un valor booleano y el programa decide qué hacer con él. La Tabla 1 resume los casos posibles.

Tabla 1

Valor que devuelve verificar_acceso() en cada situación

Situación	Valor devuelto
La persona gana dos rondas de tres	`True`
La persona cierra la ventana del CAPTCHA	`False`
La persona queda bloqueada y cierra la ventana	`False`
El CAPTCHA no se puede cargar (por ejemplo, imágenes dañadas)	`False`

El último caso tiene una razón de ser. Si las imágenes no se pueden cargar, el CAPTCHA muestra un error y niega el acceso en lugar de dejar pasar a la persona; dicho de otra forma, falla de forma cerrada. Algo parecido ocurre con el sorteo: el símbolo de cada ronda y el orden de los botones se eligen con `secrets.SystemRandom`, que toma los números aleatorios del sistema operativo y no de una semilla que un programa automatizado pueda

predecir (Python Software Foundation, s.f.-a).

La función recibe además el parámetro `anunciar_acceso`. Por defecto vale `True` y hace que, después del mensaje «Validación completada correctamente», aparezca «Acceso concedido». Conviene pasarlo en `False` cuando el programa ya tiene su propia pantalla de bienvenida, para no mostrar dos mensajes seguidos.

Opción 1: importar captcha.py

Es la forma recomendada, porque el CAPTCHA queda en un archivo aparte y el programa protegido casi no cambia. Solo hacen falta dos pasos:

1. Copiar `captcha.py` en la misma carpeta que el programa que se quiere proteger.
2. Llamar a `verificar_acceso()` al inicio del programa, antes de crear la ventana principal o de ejecutar cualquier parte protegida.

En un programa con Tkinter, el resultado se ve así:

```
import sys
import tkinter as tk

from captcha import verificar_acceso

if not verificar_acceso():
    sys.exit(0)

root = tk.Tk()
root.title("Mi aplicación")
tk.Label(root, text="Contenido protegido").pack(padx=40, pady=40)
root.mainloop()
```

Si el programa organiza su arranque dentro de una función `main()`, la llamada va al comienzo de esa función. Este mismo patrón sirve para programas de consola o con otras interfaces gráficas, porque el CAPTCHA abre y cierra su propia ventana y después le devuelve el control al código:

```
from captcha import verificar_acceso


def main() -> None:
    if not verificar_acceso(anunciar_acceso=False):
        print("Acceso denegado.")
        return

    print("Bienvenido al sistema.")
    # ... resto del programa


if __name__ == "__main__":
    main()
```

Cuando `captcha.py` se guarda en una subcarpeta, por ejemplo `seguridad/`, solo cambia la importación: `from seguridad.captcha import verificar_acceso`.

Opción 2: pegar el código dentro del programa

Esta opción sirve cuando se quiere entregar un único archivo `.py`, sin depender de `captcha.py`. Pide más cuidado que la anterior, porque el orden de las partes dentro del archivo decide si el CAPTCHA funciona o no. Por eso los pasos se deben seguir en el orden en que aparecen.

Paso 1. Agregar la importación de anotaciones

La primera línea del archivo tiene que ser la siguiente:

```
from __future__ import annotations
```

Python solo acepta esta instrucción al principio del archivo, antes de cualquier otro import. Gracias a ella, las anotaciones de tipos del CAPTCHA también funcionan en versiones de Python anteriores a la 3.10.

Paso 2. Agregar los imports del CAPTCHA

Debajo de los imports que el programa ya tenga se agregan los que usa el CAPTCHA. Los que ya existan no se repiten.

```
import base64
import json
import math
import secrets
import sys
import tkinter as tk
from collections.abc import Callable
from datetime import datetime, timedelta
from io import BytesIO
from pathlib import Path
from tkinter import messagebox

try:
    from PIL import Image, ImageTk
except ImportError:
    Image = None
    ImageTk = None
```

Paso 3. Copiar las partes correctas de captcha.py

No todo `captcha.py` debe pasar al programa. Algunas partes solo existen para que el archivo se pueda ejecutar por sí solo, y si se copian terminan abriendo ventanas que el programa no necesita. La Tabla 2 indica qué se copia y qué no.

Tabla 2

Partes de captcha.py que se copian al programa

Parte	**¿Se copia?**	**Motivo**
Constantes, desde `TITULO_CAPTCHA` hasta `AZAR`	Sí	Definen las reglas, los tiempos, los colores y la ruta del bloqueo.
Clases `GestorBloqueo`, `PartidaCaptcha` y `CaptchaApp`	Sí	Son el bloqueo persistente, el marcador de la serie y la ventana.
Funciones `cargar_imagenes()` y `verificar_acceso()`	Sí	Cargan las imágenes y abren la puerta de acceso.
Diccionario `IMAGENES_CAPTCHA` completo	Sí	Contiene las tres imágenes en base64.
Clase `ProtectedApp`	No	Es solo la ventana de demostración del archivo original.
Función `main()` y el bloque `if __name__ == "__main__":`	No	Sirven para ejecutar `captcha.py` por separado.

Paso 4. Pegar el código antes de la ventana principal

Todo lo copiado se pega antes de la línea que crea la ventana principal del programa, y justo después se agrega la verificación:

```
# ... constantes, clases y funciones del CAPTCHA ...
# ... IMAGENES_CAPTCHA = { ... } completo ...

if not verificar_acceso():
    sys.exit(0)

root = tk.Tk()
# ... el programa sigue igual que antes
```

El diccionario `IMAGENES_CAPTCHA` tiene que quedar arriba de la llamada a `verificar_acceso()`. Cuando la llamada está escrita a nivel de módulo, como en el ejemplo, Python la ejecuta en el momento en que llega a ella; si las imágenes todavía no se han definido, el programa se detiene con un `NameError`.

Paso 5. Revisar nombres repetidos

El CAPTCHA define nombres como `FUENTE`, `NOMBRES`, `AZAR`, `VENCE_A` o `cargar_imagenes`. Si el programa ya usa alguno para otra cosa, uno de los dos reemplaza al otro sin ningún aviso, y el error aparece después en un lugar que no tiene nada que ver. Antes de dar por terminada la integración vale la pena buscar esos nombres en el archivo y renombrar los que choquen.

Orden final del archivo

Al terminar, el archivo debe quedar con este orden, de arriba hacia abajo:

```
from __future__ import annotations      # 1. siempre la primera línea
import ...                              # 2. imports del programa y del CAPTCHA

# ... funciones propias del programa ... (sin cambios)

# ... constantes, clases y funciones del CAPTCHA ...
IMAGENES_CAPTCHA = {...}                # 3. imágenes embebidas

if not verificar_acceso():              # 4. la puerta de acceso
    sys.exit(0)

root = tk.Tk()                          # 5. la ventana principal, igual que antes
# ...
root.mainloop()
```

Reglas para que funcione correctamente

Sea cual sea la opción elegida, hay cuatro condiciones que la integración tiene que cumplir. La primera es la que más se pasa por alto.

- **Llamar al CAPTCHA antes de crear el propio** `tk.Tk()`. El CAPTCHA crea y destruye su propia ventana raíz. Si la ventana del programa ya existiera, la persona podría verla y usarla mientras resuelve el desafío, y la puerta de acceso perdería su sentido.
- **Llamarlo desde el hilo principal.** Tkinter espera que la interfaz se maneje desde el hilo que creó el intérprete de Tcl, así que el CAPTCHA no debe abrirse desde un hilo secundario (Python Software Foundation, s.f.-b).
- **Tener en cuenta dónde se guarda el bloqueo.** El archivo `data/lock_state.json` se crea junto al archivo que contiene el CAPTCHA: junto a `captcha.py` en la opción 1, o junto al programa en la opción 2. Si varios programas comparten esa carpeta, también comparten el bloqueo.
- **No copiar imágenes por separado.** Ya van dentro del código; basta con que el bloque `IMAGENES_CAPTCHA` esté completo.

Verificación de la integración

Antes de entregar el programa conviene probar estas cuatro situaciones a mano:

1. Al ejecutar el programa, lo primero que aparece es la ventana del CAPTCHA, y la ventana principal todavía no se ve.
2. Si se cierra el CAPTCHA con la X, el programa termina sin abrir la parte protegida.
3. Si se ganan dos rondas, aparecen los mensajes de validación y de acceso, y después se abre el programa.
4. Si se pierden dos rondas, aparece la cuenta regresiva de cincuenta segundos; al llegar a cero se puede volver a intentar.

Para repetir la prueba del bloqueo sin esperar, basta con borrar el archivo `data/lock_state.json`.

Personalización

Las reglas del CAPTCHA están en constantes al inicio del código, así que se pueden ajustar sin tocar la lógica. La Tabla 3 muestra las principales.

Tabla 3

Constantes configurables del CAPTCHA

Constante	Valor por defecto	Efecto
`TIEMPO_RONDA`	`15`	Segundos por ronda.
`ACIERTOS_PARA_VALIDAR`	`2`	Rondas ganadas necesarias para validar.
`ERRORES_PARA_BLOQUEAR`	`2`	Errores que activan el bloqueo.
`DURACION_BLOQUEO`	`timedelta(seconds=50)`	Duración del bloqueo temporal.
`PAUSA_ENTRE_RONDAS_MS`	`1000`	Pausa para mostrar el resultado antes de seguir.

`RONDAS_MAXIMAS` no se edita a mano: se calcula a partir de los aciertos y los errores, de modo que con los valores por defecto la serie nunca pasa de tres rondas.

Solución de problemas

Tabla 4

Errores frecuentes al integrar el CAPTCHA y cómo resolverlos

Síntoma	Causa y solución
`NameError: name 'IMAGENES_CAPTCHA' is not defined`	La llamada a `verificar_acceso()` quedó arriba de las imágenes. Mover el bloque `IMAGENES_CAPTCHA` antes de la llamada.
`SyntaxError: from __future__ imports must occur at the beginning of the file`	La línea `from __future__ import annotations` no es la primera del archivo. Subirla al inicio.
`ModuleNotFoundError: No module named 'captcha'`	En la opción 1, `captcha.py` no está en la misma carpeta que el programa que se ejecuta.
Mensaje «No fue posible cargar el CAPTCHA»	El bloque `IMAGENES_CAPTCHA` quedó incompleto al copiarlo. Volver a copiarlo entero.
La ventana principal aparece junto con el CAPTCHA	Se creó `tk.Tk()` antes de llamar a `verificar_acceso()`. Mover la llamada arriba.
El programa queda bloqueado durante las pruebas	Esperar los cincuenta segundos o borrar `data/lock_state.json`.
`No module named '_tkinter'` (Linux)	Instalar Tkinter con `sudo apt install python3-tk`.

Referencias

Python Software Foundation. (s.f.-a). *secrets — Generate secure random numbers for managing secrets*. Python documentation. https://docs.python.org/3/library/secrets.html

Python Software Foundation. (s.f.-b). *tkinter — Python interface to Tcl/Tk*. Python documentation. https://docs.python.org/3/library/tkinter.html